A PROJECT REPORT ON

CS23531-WEB PROGRAMMING

ECOWISE AI – SMART ENERGY ADVISOR

A RULE-BASED AI PLATFORM FOR APPLIANCE ENERGY ANALYSIS, EXPLAINABLE RECOMMENDATIONS AND COST OPTIMIZATION

A PROJECT REPORT

Submitted by

241001289– G R TEJNI



BONAFIDE CERTIFICATE

Certified that this project report “ECOWISE AI – SMART ENERGY ADVISOR” is the bonafide work of “TEJNI G R” who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO. TITLE **PAGE NO.**

ABSTRACT 4

1. INTRODUCTION 5

2. LITERATURE REVIEW 7

3. PROPOSED SYSTEM 10

3.1 ARCHITECTURE DIAGRAM 10

3.2 WORKFLOW OF THE SYSTEM 11

3.3 TECHNOLOGY STACK 12

4. MODULES DESCRIPTION 13

4.1 USER INTERFACE AND DASHBOARD MODULE 13

4.2 AUTHENTICATION AND SECURITY MODULE 13

4.3 INPUT VALIDATION MODULE 14

4.4 ENERGY AND TARIFF CALCULATOR MODULE 14

4.5 RULE-BASED REASONING MODULE 15

4.6 CSP AND BACKTRACKING MODULE 16

4.7 HILL CLIMBING OPTIMIZATION MODULE 16

4.8 RESULT, WHAT-IF AND HISTORY MODULE 17

5. IMPLEMENTATION AND RESULTS 19

5.1 EXPERIMENTAL SETUP 19

5.2 RESULTS 21

6. CONCLUSION AND FUTURE WORK 27

7. REFERENCES 29

ABSTRACT

Households usually know their total electricity bill but not how much each appliance contributes to it, and generic energy-saving tips rarely match real usage. EcoWise AI is a rule-based artificial intelligence web platform that estimates appliance-level energy consumption and cost, explains inefficient usage through transparent IF–THEN rules, and searches for a cheaper usage plan that still respects the user's comfort. The system is built with Python, Flask, HTML, CSS and JavaScript. Its AI core combines an energy and tariff calculator, thirteen explainable rules, a constraint satisfaction problem (CSP) solved by backtracking to obtain valid starting states, and Hill Climbing with swap moves and random restarts to find the lowest-cost valid plan. On a sample eleven-appliance home consuming 672.3 kWh per month, the system estimated a bill of ₹5,379 and recommended a plan that keeps 80.2% of the user's comfort while reducing the bill to ₹2,827, a saving of ₹2,552 per month (47.4%) and 261.6 kg of CO_2. Accounts, saved history, slab tariffs, a what-if simulator and CSV export are also provided, and twenty automated tests verify the logic.

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Electricity is one of the largest recurring expenses of a modern household, and its generation is a significant source of greenhouse-gas emissions. In India, residential consumers receive a single monthly bill that shows total units consumed but does not reveal which appliance consumed how many units. Without this visibility, users find it difficult to decide where to cut usage, and generic advice such as “switch off unused devices” is often too vague to act upon.

Artificial Intelligence offers a natural way to bridge this gap. Knowledge-based reasoning can turn a list of appliances and their usage hours into specific, explainable advice, and search algorithms can explore many alternative usage patterns far faster than a person could by hand. EcoWise AI applies these ideas from the Foundations of Artificial Intelligence course – rule-based reasoning, constraint satisfaction, backtracking and local search – to the practical problem of reducing household electricity cost.

1.2 PROBLEM STATEMENT

Users may know the total electricity bill but lack clear visibility into individual appliance consumption. Generic energy-saving tips may not match the actual usage pattern, and comparing alternative usage options manually takes considerable time. The problem addressed by this project is therefore to build an intelligent advisor that (i) estimates appliance-level energy use and cost, (ii) explains inefficient usage with transparent rules, and (iii) finds a lower-cost usage plan that respects appliance-specific and comfort constraints.

1.3 OBJECTIVES

- **Calculate –** Estimate daily and monthly energy consumption in kWh and the corresponding electricity cost, including standby power.
- **Explain –** Use transparent IF–THEN rules to generate understandable recommendations that show the exact condition that fired.
- **Validate –** Apply appliance-specific usage constraints (minimum hours, comfort target, optional daily cap) before optimization.
- **Optimize –** Use Hill Climbing to search for a lower-cost valid usage state.
- **Compare –** Display before/after cost with estimated daily, monthly and yearly savings and CO_2 avoided.
- **Present –** Provide a clear, responsive web dashboard with secure user accounts and saved history.

1.4 SCOPE OF THE PROJECT

The prototype works on user-entered appliance data (name, quantity, rated power in watts, hours of use per day, minimum acceptable hours, priority and standby power). It supports a flat tariff (default ₹8 per kWh) and an illustrative slab tariff. All savings are estimates that depend on real usage and actual electricity rates. The system does not read smart-meter data; this is listed as future work in Chapter 6.

1.5 ORGANIZATION OF THE REPORT

Chapter 2 reviews related work. Chapter 3 presents the proposed system, its architecture and technology stack. Chapter 4 describes each module in detail. Chapter 5 describes the implementation, experimental setup and results. Chapter 6 concludes the report and lists future enhancements, and Chapter 7 lists the references.

CHAPTER 2
LITERATURE REVIEW

This chapter summarises the body of work that informed the design of EcoWise AI: feedback-based energy saving, appliance-level energy analysis, knowledge-based (rule-based) systems, constraint satisfaction and local search.

2.1 FEEDBACK AND APPLIANCE-LEVEL VISIBILITY

Darby (2006) reviewed studies on energy feedback and concluded that giving households clearer, more frequent and more disaggregated information about their consumption can help them reduce it. Ehrhardt-Martinez, Donnelly and Laitner (2010) reached a similar conclusion from a large review of residential feedback programmes, noting that the benefit depends strongly on how actionable the feedback is. These findings motivate EcoWise AI's emphasis on appliance-wise breakdowns, an efficiency score and concrete recommendations instead of a single bill figure.

2.2 NON-INTRUSIVE LOAD MONITORING

Hart (1992) introduced non-intrusive appliance load monitoring, which estimates the usage of individual appliances from a single household-level meter reading. Armel et al. (2013) discussed whether disaggregation is the "holy grail" of energy efficiency and highlighted the value of appliance-level information for savings. Kelly and Knottenbelt (2015) applied neural networks to the same problem. These approaches need metering hardware or large datasets; EcoWise AI instead takes a lightweight, user-entered approach so that a student-level prototype can provide appliance-level estimates without sensors, and the estimate can be cross-checked against the user's actual bill through rule R12.

2.3 RULE-BASED AND EXPLAINABLE REASONING

Rule-based expert systems such as MYCIN (Buchanan and Shortliffe, 1984) showed that expert knowledge written as IF–THEN rules can produce advice whose reasoning is easy for the user to inspect. Russell and Norvig (2021) describe knowledge-based agents and rule-based inference as a core part of AI. Following this tradition, EcoWise AI encodes energy-saving knowledge as thirteen rules and shows the matched IF condition and THEN advice for every finding, so the user can see exactly why a recommendation was made.

2.4 CONSTRAINT SATISFACTION AND BACKTRACKING

A constraint satisfaction problem consists of variables, domains and constraints, and backtracking search with early pruning is the standard way to solve it (Mackworth, 1977; Dechter, 2003; Russell and Norvig, 2021). EcoWise AI models the hours of use of each appliance as variables, the allowed hour values as domains, and appliance limits, a comfort target and an optional daily cap as constraints. Backtracking with bound-based pruning is used to find valid starting states.

2.5 LOCAL SEARCH AND HILL CLIMBING

Hill Climbing is a local search algorithm that repeatedly moves to the best neighbouring state until no neighbour is better; it is simple and fast but can stop at local optima, which is commonly mitigated by random restarts (Selman and Gomes, 2006; Russell and Norvig, 2021). EcoWise AI uses steepest-descent Hill Climbing on the monthly bill, adds “swap” moves that cut one appliance and give time back to another, and uses seeded random restarts so that results are repeatable.

2.6 RESEARCH GAP AND CONTRIBUTION

Existing work either needs metering infrastructure or gives generic advice. EcoWise AI contributes a transparent, software-only advisor that (i)

computes appliance-level cost, (ii) explains findings with visible rules, and (iii) combines CSP and Hill Climbing to produce a concrete, constraint-respecting plan with before/after savings, delivered through an easy web dashboard.

CHAPTER 3
PROPOSED SYSTEM

EcoWise AI is a four-layer web application. The user enters appliance details in the browser; the Flask server validates them and calls the AI engine; the engine computes energy and cost, applies the rule base, and optimizes usage; and the result is returned as JSON and displayed as tables, charts and recommendations.

3.1 ARCHITECTURE DIAGRAM

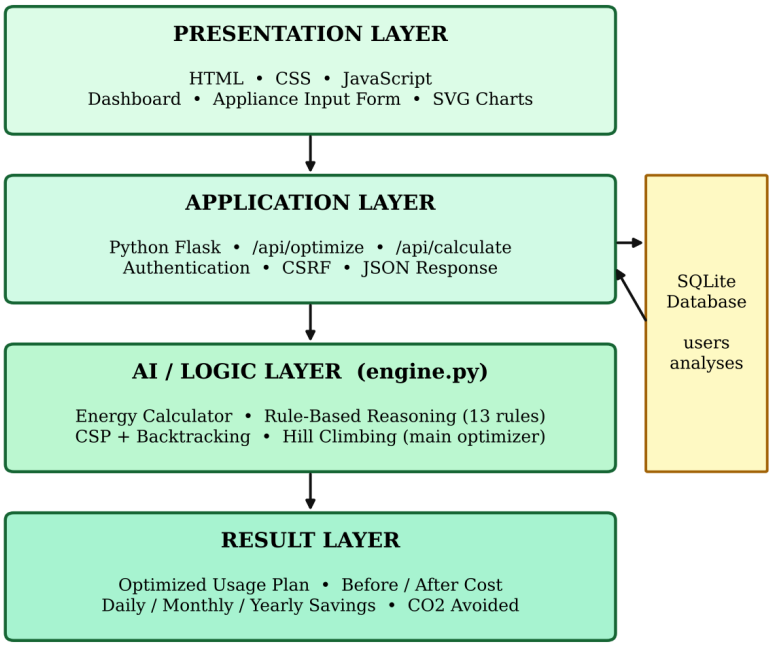


Figure 3.1 Architecture of EcoWise AI

- **Presentation layer –** HTML, CSS and JavaScript provide the dashboard, appliance input form, what-if sliders and offline SVG charts.
- **Application layer –** Python Flask handles routes, authentication, CSRF protection, the /api/optimize and /api/calculate endpoints and JSON responses.

- **AI / logic layer –** engine.py contains the energy calculator, the rule-based reasoning, CSP with backtracking, and Hill Climbing (the main optimization algorithm).
- **Result layer –** The optimized plan, before/after cost and estimated daily, monthly and yearly savings are returned to the user.

A lightweight SQLite database stores user accounts and saved analyses. A MySQL database was part of the conceptual design; in the implemented prototype SQLite is used because it needs no separate setup.

3.2 WORKFLOW OF THE SYSTEM

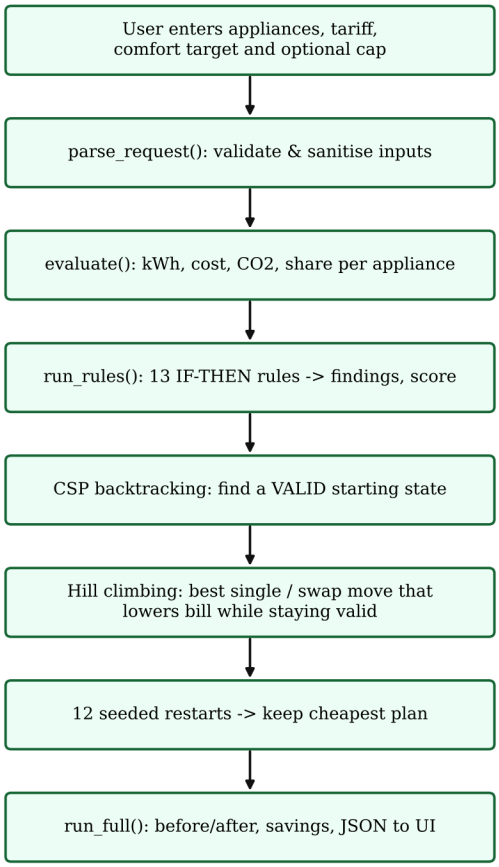


Figure 3.2 Processing flow of an Analyze & Optimize request

3.3 TECHNOLOGY STACK

Table 3.1 Technology stack of the prototype

Layer	Technology	Purpose

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript	Dashboard, appliance inputs, SVG charts, what-if simulator and result display.
Backend	Python 3 + Flask	Routes, request handling, authentication, CSRF and the JSON API.
AI / Logic	Rule-based reasoning, CSP, Backtracking, Hill Climbing	Energy calculations, validation, recommendations and optimization.
Data	SQLite (via Python sqlite3)	User accounts (hashed passwords) and saved analyses / history.
Security	Werkzeug password hashing, CSRF tokens, secure session cookies	Protect accounts and API calls.
Testing	Python unittest	20 automated tests for the engine and the web application.

Only one external Python package, Flask, is required (requirements.txt). The charts are drawn as inline SVG by charts.js, so no external charting library or internet connection is needed.

CHAPTER 4
MODULES DESCRIPTION

The system is divided into eight modules. The AI-related modules (4.3 to 4.7) are implemented in engine.py, and the interface, security and persistence modules in app.py, the templates and the static JavaScript files.

4.1 USER INTERFACE AND DASHBOARD MODULE

The dashboard (dashboard.html, dashboard.js, charts.js, style.css) lets the user pick appliances from a library of 20 presets (air conditioner, fan, LED and incandescent bulbs, refrigerator, television, washing machine, geyser, router, laptop, pump and others), add custom appliances, or load a sample home. For each appliance the user can edit quantity, watts, hours per day, minimum hours, priority (1 = easy to give up … 5 = essential) and standby watts. Tariff settings include flat rate or slab billing, the comfort level to keep (50–100%), an optional daily kWh cap and the last bill's units. After analysis the dashboard shows KPI cards, a donut chart of the energy share, the efficiency score, rule findings, the optimized plan, before/after bars, the Hill-climbing progress chart and a what-if simulator. The interface is responsive and supports dark and light themes, CSV export and a print view.

4.2 AUTHENTICATION AND SECURITY MODULE

Users can register, log in, log out, change their password and delete their account. The following protections are implemented in app.py:

- Passwords are stored only as salted hashes generated by Werkzeug (generate_password_hash / check_password_hash).
- Every POST, PUT, PATCH and DELETE request must carry a valid CSRF token (form field or X-CSRF-Token header), compared in constant time.
- A login lockout is applied after 5 failed attempts for 300 seconds.

- Session cookies are HttpOnly and SameSite=Lax; responses carry X-Content-Type-Options, X-Frame-Options and Referrer-Policy headers; API responses are not cached.
- The post-login redirect target is checked to block open-redirect attacks, and request bodies are limited to 256 KB.
- Each user can read only their own saved analyses.

4.3 INPUT VALIDATION MODULE

The function parse_request() sanitises every request before it reaches the AI logic. It requires at least one and at most 30 appliances, trims names to 60 characters, and range-checks numeric fields (quantity 1–50, watts 0.1–10 000, hours 0–24, priority 1–5, standby 0–100 W, tariff rate 0.5–100, comfort 50–100%). NaN and infinite values are rejected, minimum hours are clamped so that they never exceed the user's current hours, and unknown categories are mapped to "other". Clear error messages are returned for every invalid field.

4.4 ENERGY AND TARIFF CALCULATOR MODULE

For an appliance with quantity q, rated power W (watts), hours of use h per day and standby power S (watts), the daily energy in kWh is

$$E_day = q \times (W \times h + S \times (24 - h)) / 1000$$

The monthly energy is E_month = 30 × E_day. The monthly cost is obtained from the Tariff class. In flat mode, Cost = E_month × rate (default ₹8 per kWh). In slab mode the units are billed progressively, each band being charged at its own rate (Table 4.1). The cost of each appliance is its share of the total energy multiplied by the total bill, and CO_2 is estimated as E_month × 0.82 kg/kWh, an approximate Indian grid emission factor.

Table 4.1 Illustrative slab tariff used by the prototype (editable in engine.py)

Units in band (per month)	Rate (₹ / unit)

Units in band (per month)	Rate (₹ / unit)
0 – 100	0.00
101 – 200	2.35
201 – 400	4.70
401 – 500	6.30
501 – 600	8.40
601 – 800	9.45
801 – 1000	10.50
Above 1000	11.55

The slab rates are illustrative and must be replaced by the rates of the user's state electricity board for real use.

4.5 RULE-BASED REASONING MODULE

The reasoning module run_rules() applies thirteen transparent IF–THEN rules (Table 4.2). Each finding stores the rule id, a severity (high, medium or low), the appliance, the exact IF text with the measured values, the THEN advice and, where possible, the estimated saving in kWh per month. An efficiency score is computed as 100 minus a penalty of 12 for each high, 6 for each medium and 3 for each low finding, and is graded A (≥ 85), B (≥ 70), C (≥ 55), D (≥ 40) or E.

Table 4.2 Rule base of EcoWise AI

ID	Rule	IF (condition)	THEN (advice)
R01	Dominant load	Appliance uses ≥ 35% of daily energy	Target this appliance first
R02	Long AC runtime	AC runs > 8 h/day	Raise set-point to 24–26 °C, use a sleep timer
R03	Long heating runtime	Heater / geyser runs > 1.5 h/day	Add a timer or auto cut-off
R04	Incandescent bulb	Bulb type is incandescent	Replace with a 9 W LED (≈ 85% less energy)
R05	Long lighting hours	Lighting runs > 8 h/day	Use switches / sensors
R06	Standby load	Total standby draw ≥ 1 kWh/month	Switch off at the socket
R07	Always-on appliance	Non-essential appliance ≥ 50 W runs 24 h/day	Check whether it can be switched off
R08	High fridge draw	Refrigerator draw > 200 W	Check door seal, star rating, age
R09	Long screen time	TV / entertainment > 6 h/day	Reduce hours or enable eco mode

ID	Rule	IF (condition)	THEN (advice)
R10	Pump runtime	Water pump > 1.5 h/day	Use a timer or float switch
R11	Slab boundary	Monthly units within 15 of the next slab	Trim usage to avoid the higher rate
R12	Bill mismatch	Estimate differs from actual bill units by > 25%	Re-check hours and missing appliances
R13	High total usage	Monthly usage > 400 kWh	Review the biggest loads first

4.6 CSP AND BACKTRACKING MODULE

Before optimizing, the system must be sure that a valid plan exists. The usage-planning task is modelled as a constraint satisfaction problem:

- **Variables –** the hours per day of every appliance.
- **Domains –** values from the appliance's minimum hours up to its current hours in steps of 0.25 h.
- **Constraints –** (C1) minimum/maximum hours of each appliance; (C2) priority-weighted comfort retention ≥ the comfort target; (C3) optional total daily energy ≤ the user's cap.

The comfort retention of a plan is Σ(priority × new hours / current hours) ÷ Σ priority. Backtracking assigns appliances in order of smallest domain first and prunes a branch as soon as (a) even the best remaining choices cannot reach the comfort target or (b) even the cheapest remaining choices would exceed the daily cap. Values closest to the current usage are tried first, so the first solution found is a valid, low-change starting state. A node limit of 150 000 protects the server from pathological inputs. If no valid plan exists, the module reports why (for example, the cap is below the minimum possible usage) instead of returning a wrong answer.

4.7 HILL CLIMBING OPTIMIZATION MODULE

Hill Climbing is the main optimization algorithm. Starting from a valid state, it evaluates all neighbouring moves and takes the best one that lowers the monthly bill by at least ₹1 while keeping every constraint satisfied. Two

kinds of moves are used: single moves (one appliance's hours ±0.25 h) and swap moves (cut one appliance by one step and give one step back to another appliance). The search stops when no valid move improves the bill. To escape poor local optima, the climb is repeated from 12 additional valid starting states generated by randomized backtracking with a fixed seed, so that results are repeatable, and the cheapest plan is kept.

```
function OPTIMIZE(apps, tariff, comfort, cap):
    start <- BACKTRACK(apps, comfort, cap)   # CSP start
    if start is None: return INFEASIBLE
    best <- CLIMB(start)
    repeat 12 times:
        s <- BACKTRACK(random order)        # new start
        run <- CLIMB(s)
        if run.cost < best.cost: best=run
    return best

function CLIMB(s):
    loop:
        move <- best valid single/swap move
        if no move: return s
        apply move
```

On small cases the result was verified against brute-force enumeration (unit test test_matches_brute_force_on_small_case) and found the true optimum.

4.8 RESULT, WHAT-IF AND HISTORY MODULE

The result layer (run_full) assembles the before and after evaluations, the plan table (hours before and after, kWh and ₹ saved per appliance), human-readable recommended changes with reasons, the savings summary (daily, monthly, yearly, percentage, kWh and CO_2), the cost trace of the search and algorithm statistics (backtracking nodes, iterations, restarts). The what-if simulator calls /api/calculate so that the user can drag sliders and see the bill change using the same tariff. Analyses are saved automatically to the user's history (SQLite table analyses), can be renamed, re-opened or deleted, and are shown in a cost-trend chart on the History page. Results can be exported as CSV or printed.

Table 4.3 JSON API of EcoWise AI (login and CSRF token required)

Method	Endpoint	Purpose
POST	/api/optimize	Analyze, apply rules and optimize
POST	/api/calculate	What-if evaluation for chosen hours
POST	/api/history	Save an analysis
GET / DELETE	/api/history/<id>	Load / delete a saved analysis

CHAPTER 5
IMPLEMENTATION AND RESULTS

5.1 EXPERIMENTAL SETUP

The project is implemented as a Flask application (app.py, 441 lines) with the AI logic in a separate, framework-independent module (engine.py, 586 lines), six HTML templates, five JavaScript files and one stylesheet. The experiments were run with Python 3.13 and Flask 3.1 on a Linux machine; the dashboard was exercised in a Chromium browser. The application starts with python app.py and is opened at http://127.0.0.1:5000. A demo account (demo@ecowise.ai) is seeded for demonstration.

The test scenario is the built-in “sample home” of eleven appliances, flat tariff of ₹8 per kWh, comfort target of 80%, no daily cap. The inputs are listed in Table 5.1.

Table 5.1 Input data for the sample home

Appliance	Qty	Watts	Hours/day	Min hrs	Priority	Standby W
Air Conditioner (1.5 ton)	1	1400	8	3	4	3
Ceiling Fan	3	75	10	5	4	0
LED Bulb (9 W)	6	9	6	3	4	0
Incandescent Bulb (60 W)	2	60	4	2	3	0
Refrigerator (avg draw)	1	130	24	24	5	0
Television (LED 43")	1	90	5	1	2	1.5
Washing Machine	1	500	1	0.5	3	1
Water Heater (Geyser)	1	2000	1.5	0.25	4	0
Wi-Fi Router	1	10	24	12	3	0
Laptop	1	60	6	2	4	1
Water Pump (0.5 HP)	1	370	1	0.5	4	0

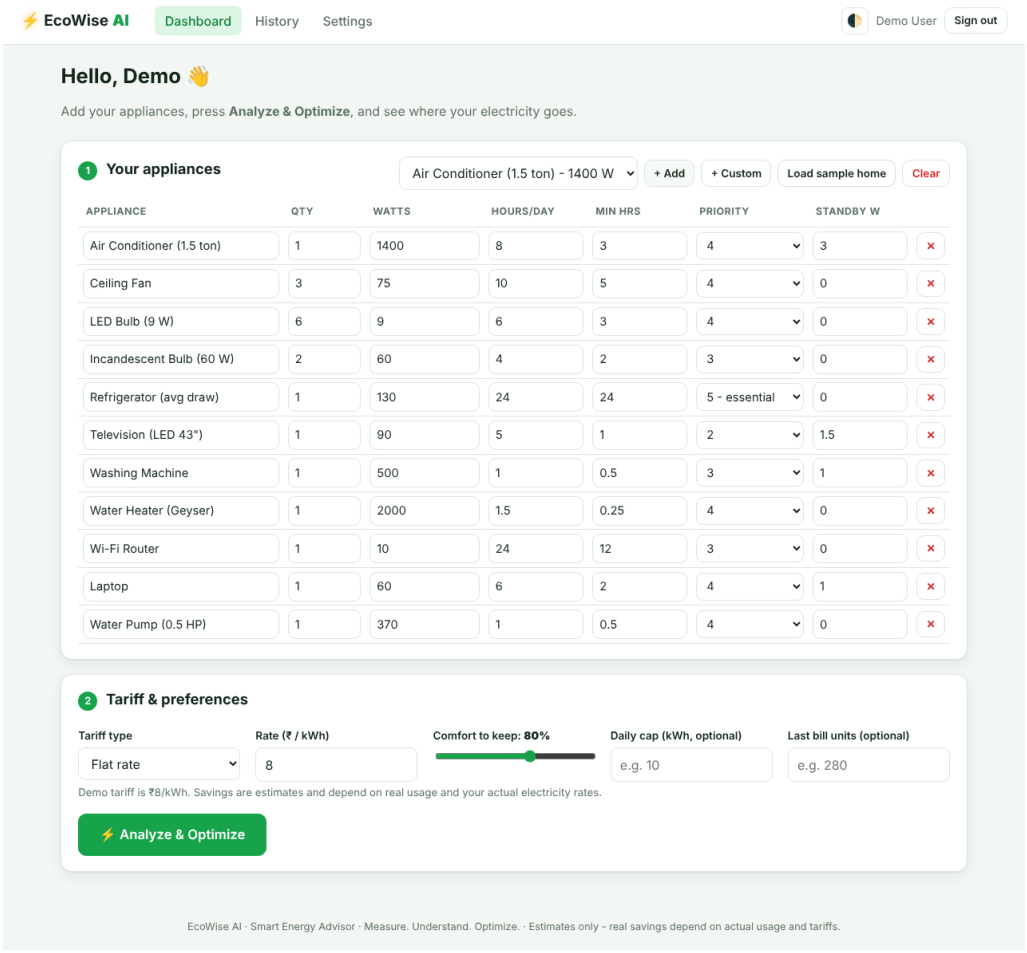


Figure 5.1 Dashboard: appliance input and tariff preferences

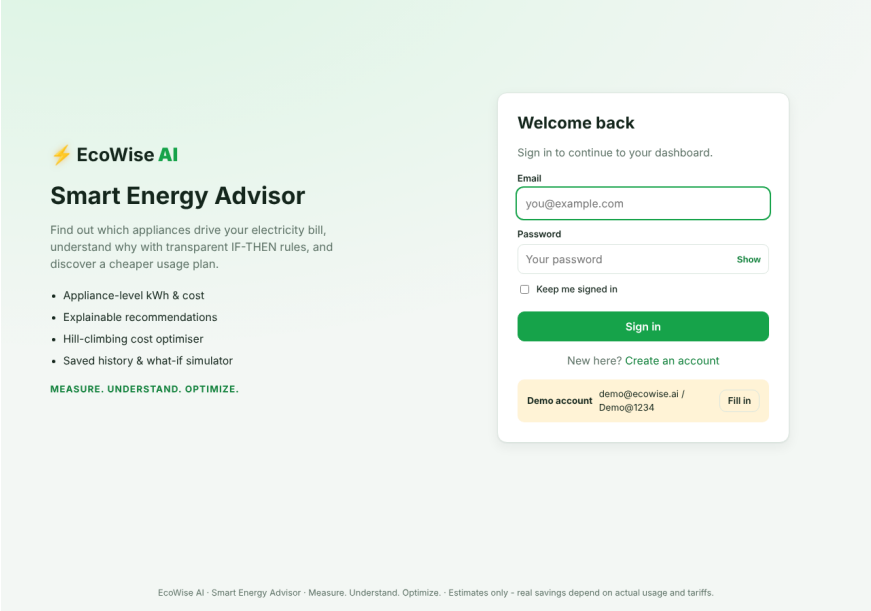


Figure 5.2 Login page

5.2 RESULTS

5.2.1 Energy and cost analysis

The calculator estimated 22.41 kWh per day (672.3 kWh per month), a monthly bill of ₹5,379 (₹179 per day) and 551 kg of CO_2 per month. The air conditioner alone accounts for 50.2% of the energy, followed by the refrigerator (13.9%), the geyser (13.4%) and the ceiling fans (10.0%). The efficiency score is 67 (Grade C).

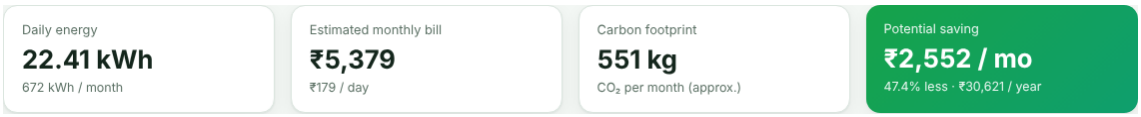


Figure 5.3 Summary cards of the analysis

Table 5.2 Appliance-wise energy and cost before optimization (flat ₹8/kWh)

Appliance	kWh/day	kWh/month	₹/month	Share
Air Conditioner (1.5 ton)	11.25	337.4	2,700	50.2%
Ceiling Fan ×3	2.25	67.5	540	10.0%
LED Bulb (9 W) ×6	0.32	9.7	78	1.4%
Incandescent Bulb (60 W) ×2	0.48	14.4	115	2.1%
Refrigerator (avg draw)	3.12	93.6	749	13.9%
Television (LED 43")	0.48	14.4	115	2.1%
Washing Machine	0.52	15.7	126	2.3%
Water Heater (Geyser)	3.00	90.0	720	13.4%
Wi-Fi Router	0.24	7.2	58	1.1%
Laptop	0.38	11.3	91	1.7%
Water Pump (0.5 HP)	0.37	11.1	89	1.7%
Total	22.41	672.3	5,379	100%

5.2.2 Rule-based recommendations

Four rules fired for the sample home: R01 (the air conditioner uses 50% of daily energy), R04 (incandescent bulbs – replacing them with 9 W LEDs would save about 12.2 kWh per month), R13 (monthly usage of 672 kWh exceeds 400 kWh) and R06 (standby draw of about 3.5 kWh per month, mainly from the air conditioner, television and washing machine). Each finding is displayed with its IF condition and THEN advice.

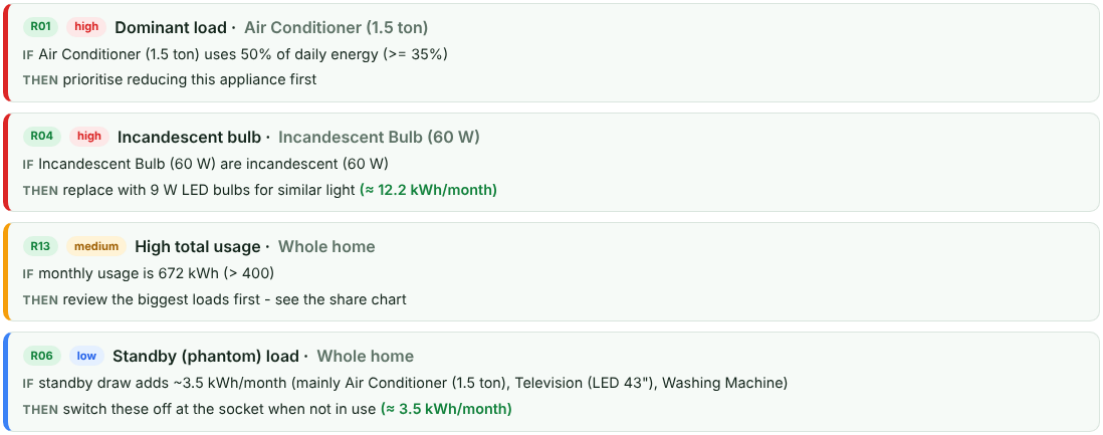


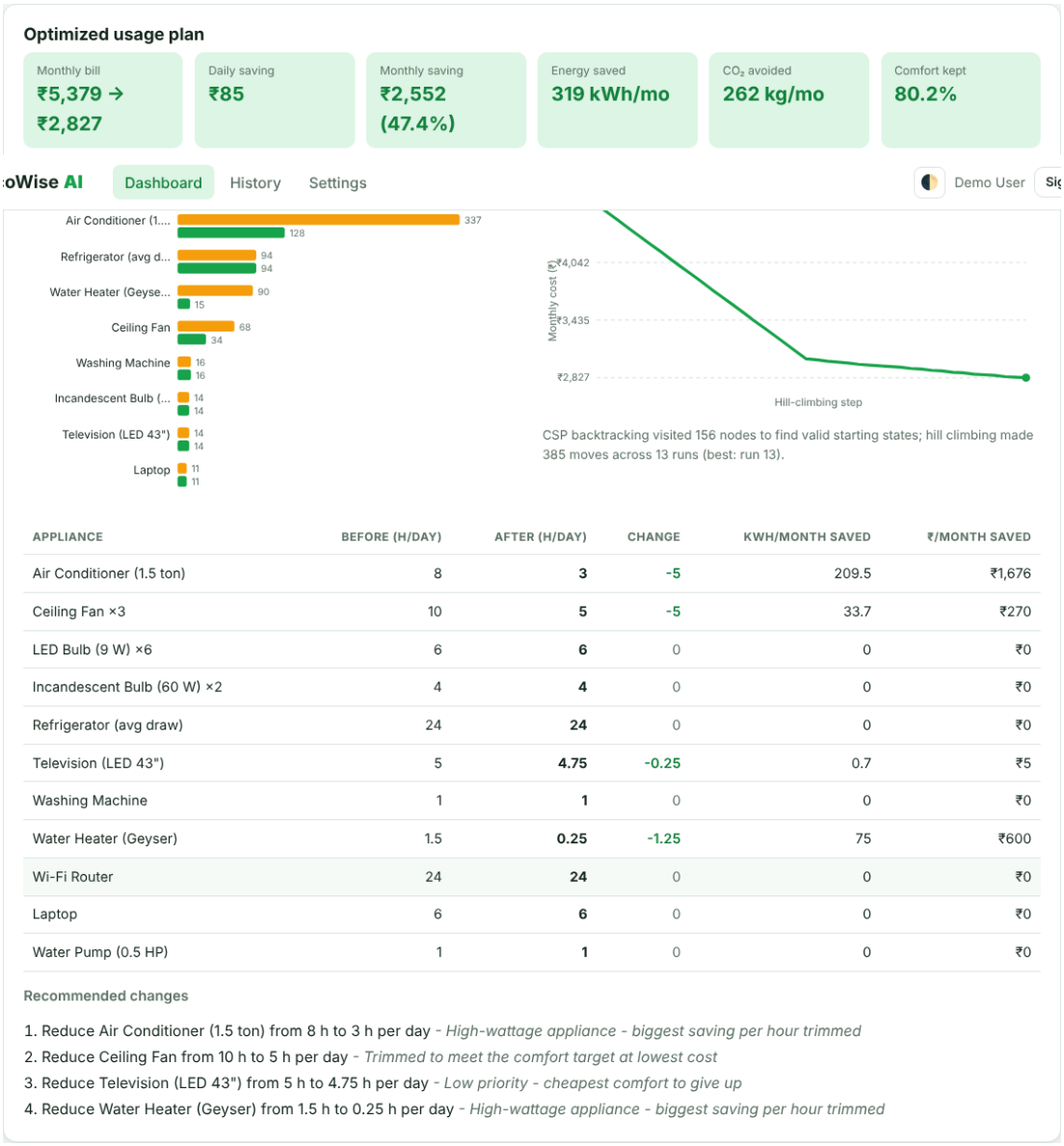
Figure 5.4 Rule-based recommendations with IF–THEN explanations

5.2.3 Optimization result

The CSP backtracking visited 156 nodes to obtain valid starting states, and Hill Climbing made 385 improving moves across 13 runs (the initial run and 12 restarts); the best run was the last restart. The optimized plan reduces the air conditioner from 8 h to 3 h, the ceiling fans from 10 h to 5 h, the television from 5 h to 4.75 h and the geyser from 1.5 h to 0.25 h per day, and leaves essential appliances such as the refrigerator, router, laptop and pump unchanged. The plan retains 80.2% of the user's comfort, which satisfies the 80% target.

Table 5.3 Changes recommended by the optimizer (flat tariff, comfort 80%)

Appliance	Before (h/day)	After (h/day)	kWh/month saved	₹/month saved
Air Conditioner (1.5 ton)	8	3	209.5	1,676
Ceiling Fan ×3	10	5	33.7	270
Television (LED 43")	5	4.75	0.7	5
Water Heater (Geyser)	1.5	0.25	75.0	600
All other appliances	unchanged	unchanged	0	0



APPLIANCE	BEFORE (H/DAY)	AFTER (H/DAY)	CHANGE	KWH/MONTH SAVED	₹/MONTH SAVED
Air Conditioner (1.5 ton)	8	**3**	-5	209.5	₹1,676
Ceiling Fan ×3	10	**5**	-5	33.7	₹270
LED Bulb (9 W) ×6	6	**6**	0	0	₹0
Incandescent Bulb (60 W) ×2	4	**4**	0	0	₹0
Refrigerator (avg draw)	24	**24**	0	0	₹0
Television (LED 43")	5	**4.75**	-0.25	0.7	₹5
Washing Machine	1	**1**	0	0	₹0
Water Heater (Geyser)	1.5	**0.25**	-1.25	75	₹600
Wi-Fi Router	24	**24**	0	0	₹0
Laptop	6	**6**	0	0	₹0
Water Pump (0.5 HP)	1	**1**	0	0	₹0

Recommended changes

1. Reduce Air Conditioner (1.5 ton) from 8 h to 3 h per day - *High-wattage appliance - biggest saving per hour trimmed*
2. Reduce Ceiling Fan from 10 h to 5 h per day - *Trimmed to meet the comfort target at lowest cost*
3. Reduce Television (LED 43") from 5 h to 4.75 h per day - *Low priority - cheapest comfort to give up*
4. Reduce Water Heater (Geyser) from 1.5 h to 0.25 h per day - *High-wattage appliance - biggest saving per hour trimmed*

Figure 5.5 Optimized usage plan, before/after energy and Hill-climbing progress

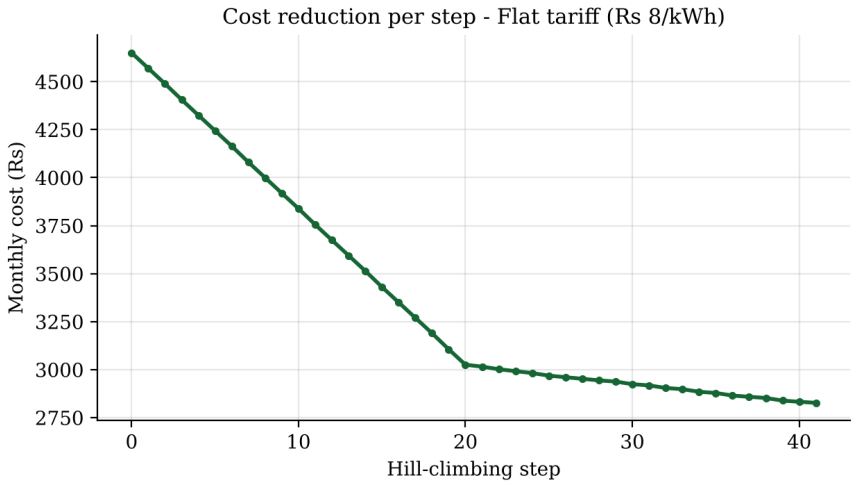


Figure 5.6 Monthly cost at each Hill-climbing step (₹4,650 → ₹2,827)

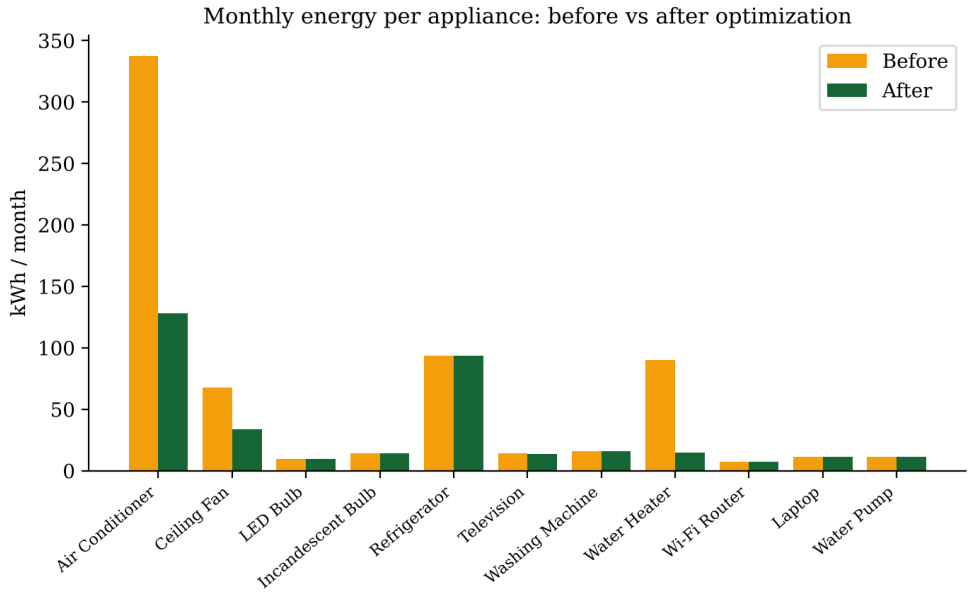


Figure 5.7 Monthly energy per appliance before and after optimization

5.2.4 Before-and-after comparison

Table 5.4 Before vs after (flat tariff)

Measure	Before	After	Saving
Energy per month (kWh)	672.3	353.4	319.0 (47.4%)
Bill per month (flat ₹8/kWh)	₹5,379	₹2,827	₹2,552 (47.4%)
Bill per day	₹179	₹94	₹85
Bill per year	₹64,545	₹33,925	₹30,621
CO_2 per month (kg)	551.3	289.7	261.6
Comfort retained	100%	80.2%	target ≥ 80%

With the illustrative slab tariff the same plan reduces the bill from ₹3,328.66 to ₹955.89, a saving of ₹2,372.77 per month (71.3%). The percentage is higher than in the flat case because the highest-priced slabs, which are charged on the largest consumption, are avoided first. This also shows why the optimizer is useful: the best plan depends on the tariff structure.

5.2.5 What-if simulator and history

The what-if simulator lets the user move sliders for each appliance and instantly see the monthly bill, saving, energy and comfort kept; its reset button returns to the optimized plan. Saved analyses are listed on the History page with a cost-trend chart.



Figure 5.8 What-if simulator

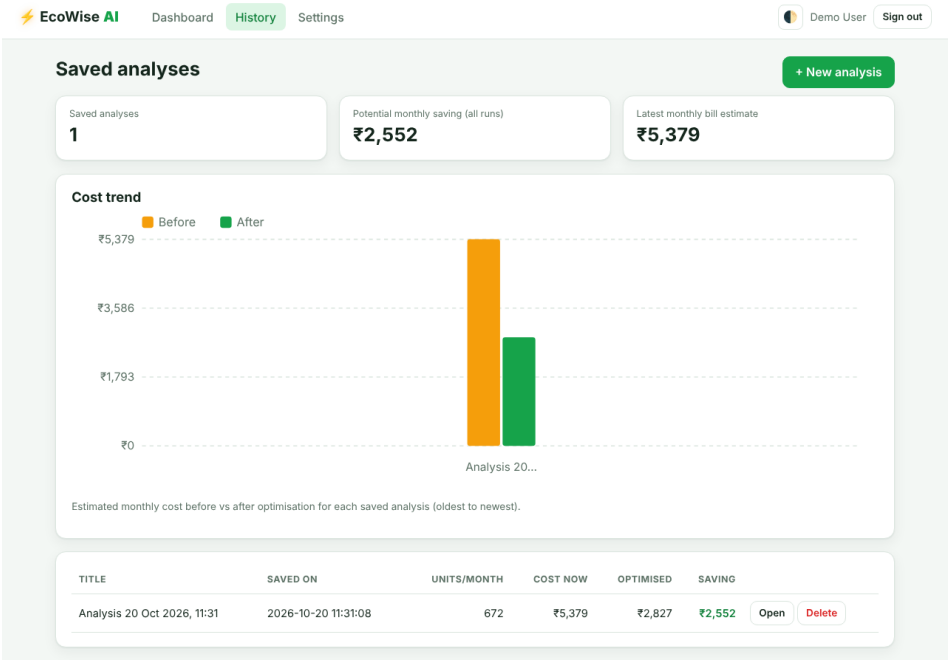


Figure 5.9 History page

5.2.6 Testing

Twenty automated unit tests (python -m unittest discover -s tests -v) all pass. They cover the energy formula, standby calculation, flat and slab tariffs, input validation, constraint satisfaction, cap handling, infeasible cases, agreement of Hill Climbing with brute force, rule firing, and the web layer (login required, CSRF protection, open-redirect blocking, registration

validation, API flow and isolation of users' histories). A complete analysis of the sample home takes about 15 ms on average on the test machine.

Table 5.5 Summary of the automated tests (20 tests in total)

Area	Test case	Expected behaviour
Energy	kWh formula; standby counts idle hours	E = q(Wh + S(24−h))/1000
Tariff	Flat tariff; slab tariff is progressive	Correct cost for each band
Validation	Empty list; bad numbers; min hours > hours	Errors returned / value clamped
CSP / search	Plan respects constraints; cap honoured	Valid plan or clear message
CSP / search	Impossible cap; full comfort target	Infeasible reported; no change
Optimality	Hill Climbing vs brute force (small case)	Same optimum cost
Rules	Rules fire for the sample home	R01, R04, R06, R13 present
Security	Pages need login; POST without CSRF; open redirect	Redirect / 400 / blocked
Accounts	Register validation and success; login / logout	Correct responses
API	Optimize → save → load → delete; user isolation	Works; other users get 404

CHAPTER 6
CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

EcoWise AI shows that classical AI techniques – explainable rules, constraint satisfaction with backtracking and Hill Climbing local search – can be combined to give practical household energy advice. The system turns a list of appliances into appliance-wise energy and cost estimates, shows why usage is inefficient with transparent IF–THEN rules, and finds a cheaper usage plan that respects appliance limits and a user-defined comfort level. On the sample home it cut the estimated bill from ₹5,379 to ₹2,827 per month (47.4%) and avoided about 262 kg of CO_2 per month, while retaining 80.2% of the user's comfort. The additions of secure accounts, saved history, slab tariffs, a what-if simulator and automated tests make the prototype a usable and reliable demonstration of the idea.

6.2 LIMITATIONS

- Inputs are entered manually by the user, so results depend on the accuracy of the entered watts and hours.
- The demo uses an assumed flat tariff of ₹8/kWh and illustrative slab rates; actual tariffs differ by state and change over time.
- Savings are estimates and depend on real usage and electricity rates; the CO_2 factor (0.82 kg/kWh) is approximate.
- Hill Climbing is a local search and does not guarantee a global optimum in every case, although random restarts reduce this risk and it matched brute force on small cases.
- There is no smart-meter integration, and the database is a single-file SQLite store suited to a prototype.

6.3 FUTURE WORK

- Replace the illustrative slabs with actual state tariff slabs and time-of-day tariffs, and read data from electricity bills.
- Connect smart-meter or IoT plug readings for automatic monitoring instead of manual entry.
- Move from SQLite to a server database such as MySQL or PostgreSQL for multi-user deployment.
- Add forecasting of future consumption and richer historical reports, and compare Hill Climbing with other search methods such as simulated annealing or A*.
- Provide mobile notifications and solar / battery what-if scenarios.

The future enhancements above are proposed features and are not claimed as implemented.

REFERENCES

Armel, K.C., Gupta, A., Shrimali, G. and Albert, A. (2013) 'Is disaggregation the holy grail of energy efficiency? The case of electricity', Energy Policy, Vol.52, pp.213–234.

Buchanan, B.G. and Shortliffe, E.H. (eds) (1984) Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project, Addison-Wesley, Reading, MA.

Bureau of Energy Efficiency, Government of India. Standards and Labelling (Star Rating) Programme for Appliances.

Central Electricity Authority, Government of India. CO2 Baseline Database for the Indian Power Sector.

Darby, S. (2006) The Effectiveness of Feedback on Energy Consumption: A Review for DEFRA of the Literature on Metering, Billing and Direct Displays, Environmental Change Institute, University of Oxford.

Dechter, R. (2003) Constraint Processing, Morgan Kaufmann, San Francisco.

Ehrhardt-Martinez, K., Donnelly, K.A. and Laitner, J.A. (2010) Advanced Metering Initiatives and Residential Feedback Programs: A Meta-Review for Household Electricity-Saving Opportunities, Report E105, American Council for an Energy-Efficient Economy, Washington, DC.

Grinberg, M. (2018) Flask Web Development: Developing Web Applications with Python, 2nd edn, O'Reilly Media, Sebastopol, CA.

Hart, G.W. (1992) 'Nonintrusive appliance load monitoring', Proceedings of the IEEE, Vol.80, No.12, pp.1870–1891.

Kelly, J. and Knottenbelt, W. (2015) 'Neural NILM: Deep neural networks applied to energy disaggregation', Proceedings of the 2nd ACM International Conference on Embedded Systems for Energy-Efficient Built Environments (BuildSys), pp.55–64.

Mackworth, A.K. (1977) 'Consistency in networks of relations', Artificial Intelligence, Vol.8, No.1, pp.99–118.

OWASP Foundation. Cross-Site Request Forgery Prevention Cheat Sheet.

Russell, S. and Norvig, P. (2021) Artificial Intelligence: A Modern Approach, 4th edn, Pearson, Hoboken, NJ.

Selman, B. and Gomes, C.P. (2006) 'Hill-climbing search', in Encyclopedia of Cognitive Science, John Wiley & Sons, Chichester.